

Compilation

Analyse Sémantique

Nicolas Delestre

Rappel sur l'analyse sémantique

But de l'analyse sémantique

- Vérifier que l'AST à un « sens », par exemple :
 - dans une opération, vérifier que les types des opérandes d'un opérateur sont bien compatibles (quite à faire quelques transtypages automatiques)
 - dans une affectation, vérifier que le type de la variable est compatible avec le type de l'expression
 - dans l'appel d'une fonction, une procédure ou une méthode, de vérifier que le nombre et les types des paramètres effectifs sont compatibles avec la signature du sous-programme en question

Comment ?

- En ajoutant des informations (attributs) aux terminaux et non terminaux de la grammaire non contextuelle de l'analyseur syntaxique et en ajoutant des règles permettant de calculer les valeurs de ces attributs ⇒ **Grammaires attribuées** (Donald Knuth et Peter Wegner)

Type de l'expression arithmétique C : 2+3.

- Extrait de la grammaire de reconnaissance d'une expression limité à l'addition :

$$\textcircled{1} \quad E \rightarrow T \mid E \text{ add } T$$

$$\textcircled{2} \quad T \rightarrow F$$

$$\textcircled{3} \quad F \rightarrow nb$$

Type de l'expression arithmétique C : 2+3.

- Extrait de la grammaire de reconnaissance d'une expression limité à l'addition :

$$\textcircled{1} E \rightarrow T \mid E \text{ add } T \quad \textcircled{2} T \rightarrow F \quad \textcircled{3} F \rightarrow nb$$

- Donc pour l'expression 2+3., on a les dérivations à gauche suivantes :

$$E \Rightarrow E \text{ add } T \Rightarrow T \text{ add } T \Rightarrow F \text{ add } T \Rightarrow nb \text{ add } T \Rightarrow nb \text{ add } F \Rightarrow nb \text{ add } nb$$

Type de l'expression arithmétique C : 2+3.

- Extrait de la grammaire de reconnaissance d'une expression limité à l'addition :

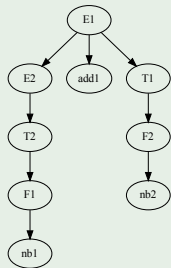
❶ $E \rightarrow T \mid E \text{ add } T$

❷ $T \rightarrow F$

❸ $F \rightarrow \text{nb}$

- Donc pour l'expression 2+3., on a les dérivations à gauche suivantes :

$E1 \Rightarrow E2 \text{ add } 1 \quad T1 \Rightarrow T2 \text{ add } 1 \quad T1 \Rightarrow F1 \text{ add } 1 \quad T1 \Rightarrow \text{nb1} \text{ add } 1 \quad T1 \Rightarrow \text{nb1} \text{ add } 1 \quad F2 \Rightarrow \text{nb1} \text{ add } 1 \quad \text{nb2}$



Type de l'expression arithmétique C : 2+3.

- Extrait de la grammaire de reconnaissance d'une expression limité à l'addition :

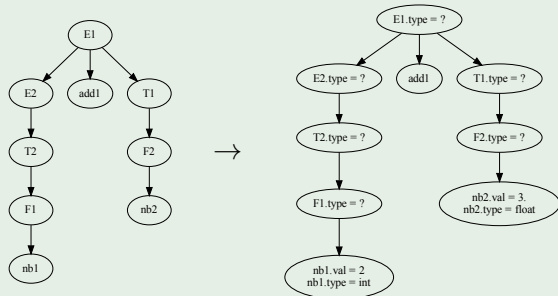
$$\textcircled{1} E \rightarrow T \mid E \text{ add } T$$

$$\textcircled{2} T \rightarrow F$$

$$\textcircled{3} F \rightarrow \text{nb}$$

- Donc pour l'expression 2+3., on a les dérivations à gauche suivantes :

$E1 \Rightarrow E2 \text{ add1 } T1 \Rightarrow T2 \text{ add1 } T1 \Rightarrow F1 \text{ add1 } T1 \Rightarrow \text{nb1} \text{ add1 } T1 \Rightarrow \text{nb1} \text{ add1 } F2 \Rightarrow \text{nb1} \text{ add1 } \text{nb2}$



Type de l'expression arithmétique C : 2+3.

- Extrait de la grammaire de reconnaissance d'une expression limité à l'addition :

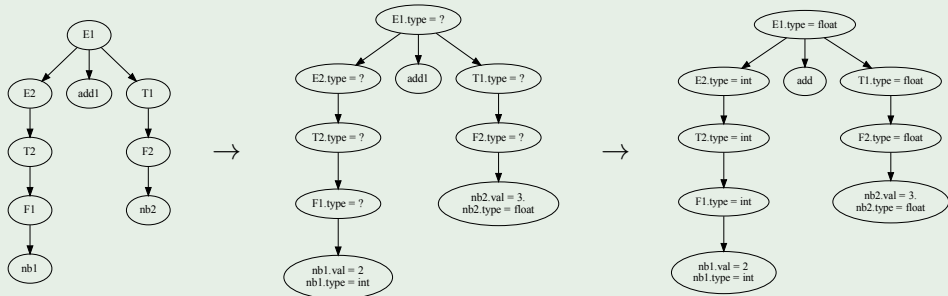
$$\textcircled{1} E \rightarrow T | E \text{ add } T$$

$$\textcircled{2} T \rightarrow F$$

$$\textcircled{3} F \rightarrow \text{nb}$$

- Donc pour l'expression 2+3., on a les dérivations à gauche suivantes :

$E1 \Rightarrow E2 \text{ add1 } T1 \Rightarrow T2 \text{ add1 } T1 \Rightarrow F1 \text{ add1 } T1 \Rightarrow \text{nb1 add1 } T1 \Rightarrow \text{nb1 add1 } F2 \Rightarrow \text{nb1 add1 nb2}$



Type des variables en C : `int a,b`

- Extraits de la grammaire de reconnaissance d'une expression :

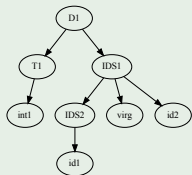
① $D \rightarrow T \text{ IDS}$

② $\text{IDS} \rightarrow id | \text{IDS virg } id$

③ $T \rightarrow int | float$

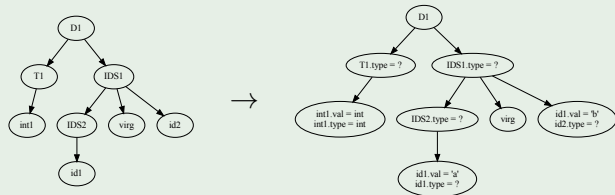
Type des variables en C : `int a,b`

- Extraits de la grammaire de reconnaissance d'une expression :
 - $D \rightarrow T \text{ IDS}$
 - $\text{IDS} \rightarrow \text{id} | \text{IDS virg id}$
 - $T \rightarrow \text{int} | \text{float}$
- Donc pour la déclaration `int a,b`, on a les dérivations à gauche suivantes :
 $D \Rightarrow T \text{ IDS} \Rightarrow \text{int IDS} \Rightarrow \text{int IDS virg id} \Rightarrow \text{int id virg id}$



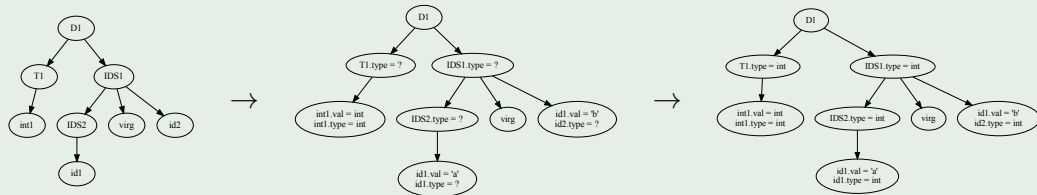
Type des variables en C : `int a,b`

- Extraits de la grammaire de reconnaissance d'une expression :
 - $D \rightarrow T \text{ IDS}$
 - $\text{IDS} \rightarrow \text{id} | \text{IDS virg id}$
 - $T \rightarrow \text{int} | \text{float}$
- Donc pour la déclaration `int a,b`, on a les dérivations à gauche suivantes :
 $D \Rightarrow T \text{ IDS} \Rightarrow \text{int IDS} \Rightarrow \text{int IDS virg id} \Rightarrow \text{int id virg id}$



Type des variables en C : `int a,b`

- Extraits de la grammaire de reconnaissance d'une expression :
 - $D \rightarrow T \text{ IDS}$
 - $\text{IDS} \rightarrow \text{id} | \text{IDS virg id}$
 - $T \rightarrow \text{int} | \text{float}$
- Donc pour la déclaration `int a,b`, on a les dérivations à gauche suivantes :
 $D \Rightarrow T \text{ IDS} \Rightarrow \text{int IDS} \Rightarrow \text{int IDS virg id} \Rightarrow \text{int id virg id}$

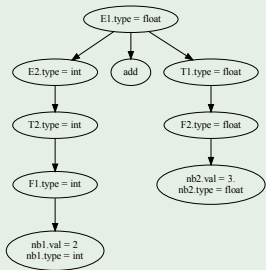


Deux catégories d'attributs 1 / 2

Attributs synthétisés

- Attribut d'un non terminal de la partie gauche d'une règle dont la valeur est calculée à partir des valeurs d'attributs des terminaux ou non terminaux (notés α^i) de la partie droite de la règle
 - Soit la règle : $A \rightarrow \alpha^1 \dots \alpha^i \dots \alpha^n$
 - On a : $A_{attr} \leftarrow f(\alpha^1_{attr}, \dots, \alpha^i_{attr}, \dots, \alpha^n_{attr})$
- Au niveau de l'AST cela signifie que la valeur de l'attribut d'un nœud ne dépend que des valeurs des attributs de ses fils

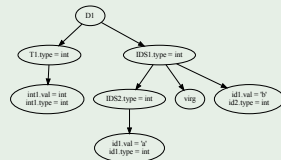
2+3.



Attribut hérité

- Attribut d'un non terminal de la partie droite d'une règle dont la valeur est calculée à partir de la valeur de l'attribut du non terminal de la partie gauche de la règle et des valeurs des attributs des autres terminaux ou non terminaux (notés α^i) de la partie droite de la règle
 - Soit la règle : $A \rightarrow \alpha^1 \dots A^i \dots \alpha^n$
 - On a : $\alpha_{attr}^i \leftarrow f(A_{attr}, \alpha_{attr}^1, \dots, \alpha_{attr}^{i-1}, \alpha_{attr}^{i+1}, \dots, \alpha_{attr}^n)$
- Au niveau de l'AST, cela signifie que la valeur de l'attribut d'un nœud dépend des valeurs d'attributs de son nœud « père » et des nœuds « frères »

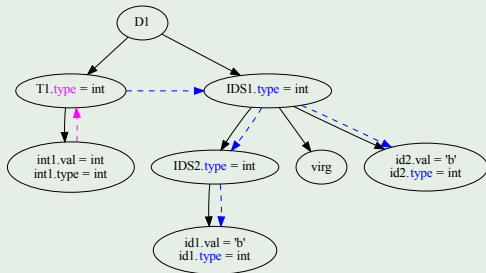
int a,b



Graphe de dépendances

- Il y a un ordre pour le calcul d'une valeur d'un attribut, graphe orienté acyclique $\Rightarrow$ ordre partiel
- Utilisation d'un tri topologique : ordre partiel $\Rightarrow$ ordre total

Type de variables en C : `int a,b`



attribut synthétisé
attribut hérité

Tri topologique :

- 1 T1
- 2 IDS1
- 3 IDS2
- 4 id1
- 5 id2

Règle sémantique

Principe

- Ajouter du code pour calculer les valeurs des attributs

Pour calculer le type d'une expression arithmétique en C

Règles de la grammaire	Règles sémantiques
$E \rightarrow T$	$E.type \leftarrow T_1.type$
$E \rightarrow E \text{ add } T$	Si $E_1.type = int$ et $T_1.type = int$ alors $E.type \leftarrow int$ sinon $E.type \leftarrow float$
$E \rightarrow E \text{ sous } T$	Si $E_1.type = int$ et $T_1.type = int$ alors $E.type \leftarrow int$ sinon $E.type \leftarrow float$
$T \rightarrow F$	$T.type \leftarrow F.type$
$T \rightarrow T \text{ mul } F$	Si $T_1.type = int$ et $F_1.type = int$ alors $T.type \leftarrow int$ sinon $T.type \leftarrow float$
$T \rightarrow T \text{ div } F$	Si $T_1.type = int$ et $F_1.type = int$ alors $T.type \leftarrow int$ sinon $T.type \leftarrow float$
$F \rightarrow nb$	$F.type \leftarrow nb.type$

Concrètement comment stocker ces attributs ?

Des classes différentes pour chaque nœud de l'AST

- Aujourd'hui le paradigme de programmation le plus répandu est celui de la POO orienté classe :
 - Les classes définissent les attributs de leurs instances
 - L'état d'un objet est défini par les valeurs de ses attributs
- On pourrait donc :
 - associer à chaque terminal et non terminal de la grammaire des classes
 - lors de l'analyse syntaxique instancier les objets

Comment ?

- En utilisant des schémas de traduction : notation permettant d'ajouter des fragments de programmes dans la partie droite d'une règle
- Ces fragments de programme sont exécutés au déclenchement des règles lors de l'analyse syntaxique permettant de créer les nœuds de l'arbre

Exemple d'un langage du paradigme impératif 1 / 3

Extrait de la grammaire

Programme → *Instructions*

Instructions → *Instruction* | *Instruction Instructions*

Instruction → *Affectation* | *Conditionnelle* | *Iteration*

Affectation → *id aff Expression*

Conditionnelle → *si Expression alors debut Instructions fin*

...

Expression → *nb* | *id* | (*Expression*) | *OperationUnaire* | *OperationBinaire*

OperationUnaire → *sous Expression* | *nonExpression*

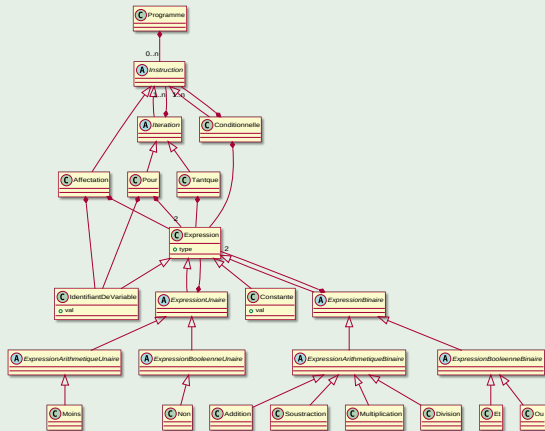
OperationBinaire → *Expression add Expression* | *Expression sous Expression* |

Expression mul Expression | *Expression div Expression* | *Expression et Expression* | *Expression ou Expression*

...

Exemple d'un langage du paradigme impératif 2 / 3

Extrait du diagramme de classes



- Lors de l'analyse syntaxique, lorsque certaines règles sont « déclenchées » :
 - des objets correspondants sont instanciés
 - des objets sont reliés

Exemple d'un langage du paradigme impératif 3 / 3

Extrait de la grammaire attribuée avec schéma de traduction en Java

- ① *Programme* $\rightarrow$ *Instructions* {return new Programme(\$1);}
- ② *Instructions* $\rightarrow$ *Instruction* {List<Instruction> l = new ArrayList<>();
l.add(\$1); return l;}
- ③ *Instructions* $\rightarrow$ *Instruction Instructions* {List<Instruction> l = new
ArrayList<>(); l.add(\$1); l.addAll(\$2); return l;}
- ...
- ④ *Affectation* $\rightarrow$ *id aff Expression* {return new Affectation(new
IdentifiantDeVariable(\$1.val),\$3);}
- ...
- ⑤ *Expression* $\rightarrow$ *nb* {return new Constante(\$1.val);}
- ⑥ *Expression* $\rightarrow$ *id* {return new IdentifiantDeVariable(\$1.val);}

Conclusion

- Dans ce cours nous avons vu que l'analyse sémantique permet de vérifier l'AST
- Cette vérification est réalisée à l'aide de :
 - l'ajout d'attributs (hérités ou synthétisés) aux terminaux et non terminaux de la grammaire
⇒ grammaire attribuée
 - l'attribution de valeurs à ces attributs ⇒ règles sémantiques et tri topologique
 - la différenciation des nœuds de l'AST ⇒ modèle orienté objet
 - l'instanciation des nœuds de l'AST ⇒ schéma de traduction